

Alla Vostra Mobile App Work Log - June 30 and July 1, 2026

Compiled summary for Tuesday, June 30, 2026 and Wednesday, July 1, 2026 - focus: filled-cart 2x2 product-grid margins, payment-overlay button and form polish, shop-step progress sliders, Stripe and Vercel payment infrastructure, card-only checkout cleanup, Stripe secure card capture without a second payment sheet, Postmark confirmation emails, and the manual account/dashboard setup needed to prove the order flow end to end.

Generated automatically for the July 1 combined artifact after reviewing the recent root-level worklog PDFs, the June 30 and July 1 Git commit chain, current mobile and server source files, local syntax and whitespace checks, Stripe/Vercel/Postmark setup outcomes from the implementation session, and the clean working tree state before this artifact was created.

Includes tracked Git work from commits b9cd6af, db4116a, f1a83f5, 89fb8ab, 2cab30, e940bc6, 1a90de5, and 7493437. Together these commits moved the Shop/payment experience from a polished front-end checkout path into a Stripe-backed mobile payment flow with a Vercel-hosted backend and Postmark confirmation email delivery.

Source Materials Processed

- Read recent root-level PDF worklogs, especially June 24 through June 27, with pdftotext to preserve the established title cadence, compact summary opening, source-material section, baseline section, bullet-heavy work sections, validation section, follow-up context, and output-artifact close.
- Reviewed the June 30 Git history from b9cd6af through 2cab30, including commit messages, changed files, patch stats, and code diffs for filled-cart grid margins, payment overlay field/icon work, center-button slider behavior, StripeProvider setup, Vercel serverless routes, and card-only PaymentIntent creation.
- Reviewed the July 1 Git history from e940bc6 through 7493437, including the Postmark webhook/email backend, mobile Stripe secure-card integration, and the later CardForm pass that restored the split card-number, expiration, and CVV presentation.
- Checked current source context in mob/app/shop.js, mob/styles/shopStyles.js, mob/app/_layout.js, mob/utils/stripePayments.js, mob/app.json, server/api/payment-sheet.js, server/api/stripe-webhook.js, server/lib/postmark.js, and server/README.md.
- Reviewed project state files alla_vostra_APP_PROJECT_SNAPSHOT.txt and alla_vostra_WEB_PROJECT_SNAPSHOT.txt as part of the June 30 chain, confirming that active app work remained in /Users/prahlin/alla_vostra/mob while payment backend work lived under /Users/prahlin/alla_vostra/server.
- Reviewed the implementation-session setup outcomes: Vercel project deployment, Stripe sandbox/payment-method configuration, Stripe webhook destination, Postmark sender/stream setup, Vercel environment variables, webhook delivery status, Postmark Activity delivery, and Gmail receipt confirmation.
- Used git log, git show --stat, git show --patch, git status, rg, sed, pdftotext, pdfinfo, node --check, npm test --prefix server, git diff --check, and a small standard-library PDF writer; no external PDF package was installed.

Current Baseline to Preserve

- Primary app work remains in /Users/prahlin/alla_vostra/mob, using Expo Router, React Native, the shared Shop cart context, and the Shop overlay sequence Products, Cart,

Contact, Delivery, Payment, and Confirmed.

- Server-side payment work now lives in /Users/prahlin/alla_vostra/server as a tiny Vercel-compatible Node backend. It keeps secret keys off the phone and exposes health, payment creation, and Stripe webhook routes.
- Stripe is installed and wired on both sides: the mobile app uses @stripe/stripe-react-native and StripeProvider, while the server uses the Stripe Node SDK to create PaymentIntents and verify webhook signatures.
- Real card capture should remain inside Stripe-native secure elements. The app may style the surrounding payment overlay, but it should not collect or store raw card numbers, expiration dates, or CVV values in ordinary React Native text fields.
- The production-shaped payment flow is card-only for now. Google Pay, Apple Pay, and PayPal remain visual/payment-method choices in the overlay, but official Stripe checkout is constrained to Debit/Credit Card until those wallet paths are deliberately implemented.
- Postmark confirmation emails are sent server-side only after Stripe confirms payment_intent.succeeded. The mobile app does not send customer emails directly and does not contain the Postmark server token.
- Vercel environment variables now carry sensitive configuration. The repo should continue to document variable names through .env.example files while never committing real Stripe, Postmark, or webhook secrets.

Main Goals We Worked Toward

- Finish the filled-cart 2x2 product-entry margin request by increasing left/right grid margins while keeping asset alignment and long divider widths synchronized.
- Make the Payment overlay feel like part of the Alla Vostra visual system: white wallet buttons, official service icons, hairline borders, taller readable form fields, and card fields that only appear after Debit/Credit Card is selected.
- Replace confusing dashed or clipped input text with readable generic text styling inside form fields, while preserving the branded typography around the fields.
- Turn the orange top/bottom center-button banners into five-step progress sliders that move through Products, Cart, Contact, Delivery, and Payment, then switch to all-green only after the user confirms the order.
- Connect Stripe carefully enough for real app-store deployment later: mobile publishable configuration, server secret-key configuration, Vercel deployment, server-side totals, payment intent creation, and mobile payment confirmation.
- Remove the second gray Stripe payment sheet/card-entry overlay by carrying the in-app secure card entry into the official Stripe confirmation call.
- Add a Postmark notification path so a successful Stripe payment produces a customer-facing confirmation email with order summary, delivery address, and payment reference.

Work Completed - Filled-Cart Grid Margins and Dividers

- Increased the filled-cart product-entry grid left and right margins by 50 percent through a dedicated cartOverlayProductGridMarginScale rather than by ad hoc offsets.
- Moved the long horizontal product divider insets to use the new product-grid left and right boundaries, making the long divider widths reflect the narrower grid instead of spanning the old broader cream-area space.
- Retied the product asset grid to the new product-grid left and right edges so the top-left, top-right, bottom-left, and bottom-right cells align horizontally inside the updated boundaries.

- Kept image and control placement based on the new asset-grid left edge, so the product image lane and counter/remove lane remain centered inside the revised 2x2 grid.
- Preserved the prior scrollbar reservation logic and row-available-width math, but made the usable product row derive from the product grid width rather than the full cream-area width.
- Left the already-sensitive filled-cart row structure intact: product name/price, Serves text, image, counter, remove button, mini dividers, long dividers, and scrollbar still operate as one coupled layout system.

Work Completed - Payment Overlay Visual Expansion

- Expanded the Payment overlay from a simple method list into a fuller payment-method surface with Google Pay, Apple Pay, PayPal, and Debit/Credit Card options.
- Added royalty-free service mark assets for Google Pay, Apple Pay, and PayPal under mob/assets/payments and rendered them inside white wallet-style buttons.
- Scaled wallet buttons from available overlay width so the three buttons fill the cream area horizontally while accounting for the existing left/right overlay margins.
- Changed wallet button surfaces to plain white and replaced heavier borders with app hairlines so the controls read as light payment controls rather than thick black blocks.
- Removed the top Payment word from the overlay so the payment surface relied on the established center-button label and form controls rather than redundant heading text.
- Kept Debit/Credit Card as the explicit card path while the wallet buttons remain separate selectable payment choices.

Work Completed - Payment Form Fields and Input Readability

- Generated credit/debit card form fields that matched the Delivery and Contact field sizing, inactive/active states, row language, and label treatment.
- Made the card fields appear only after tapping Debit/Credit Card, preventing the payment screen from being crowded until the user chooses the card path.
- Split the cardholder name field into First name and Last Name, matching the Contact overlay convention rather than using one long combined cardholder field.
- Added the Billing address matches delivery address checkbox under the bottom row of payment fields, then adjusted the label to wrap instead of bleeding into nearby buttons.
- Increased form-field height across overlays by 25 percent, then refined card-field font sizing and line height so typed characters became visible and did not clip or vertically scroll inside their fields.
- Changed the inputted text font to a generic readable font inside fields only, resolving the Android/Expo Go issue where typed characters displayed as tiny dashes or empty spaces under the branded TT Fors font.
- Added Issuer labeling and selection/readout behavior, with issuer options VISA, MASTERCARD, and AMEX, then moved Issuer to the left and its dropdown/readout box to the right with matched field height.
- Refined card-number, first-name, and issuer/dropdown alignment so right edges and row proportions matched the surrounding form grid.

Work Completed - Shop Center Button Progress Slider

- Transformed the orange top and bottom bands inside the Shop center button into a progress-slider treatment that mimics the app header slider behavior during main-screen navigation.
- Set the slider to five portions for the five shop-related steps: Products, Cart, Contact, Delivery, and Payment.

- Animated the active orange segment between portions when the user navigates through the order flow.
- Kept the center button named Payment while the user is still on the payment/prompt phase, so it does not switch to Confirmed until the user presses Yes to officially place the order.
- After Yes, changed the center-button top/bottom bands, left side button, and right side button to green, preserving the right triangle button visibility/clickability where requested.
- Removed the white triangles from the Confirmed center button state so the final center button reads as a green completion state rather than an active navigation control.
- Kept the left/right side buttons and center button geometry responsive, preserving the established overlap, side-button framing, and sticky-cart spacing while the band logic changed.

Work Completed - Stripe and Vercel Backend Foundation

- Created the Vercel-ready server project under server with package.json, package-lock.json, README.md, .env.example, a health route, and a payment-sheet route.
- Built server/api/payment-sheet.js to create Stripe PaymentIntents with server-controlled product prices for Piccola, Sei Perfetto, and Buon Natale rather than trusting amounts from the app.
- Added order payload parsing, subtotal, delivery fee, tax, grand-total math, contact email handling, shipping details, and Stripe metadata containing order items and order summary details.
- Added mobile Stripe configuration in mob/utils/stripePayments.js, including publishable key, backend payment route URL, merchant identifier, live/test mode detection, return URL, URL scheme, and configuration-problem messages.
- Wrapped the app with StripeProvider in mob/app/_layout.js and added Stripe plugin configuration to mob/app.json with merchant.com.allavostra and Google Pay enabled for eventual production builds.
- Changed the Yes confirmation path so it creates a server PaymentIntent, initializes mobile payment processing, handles errors, shows processing state, and only advances to the green confirmed order state after Stripe succeeds.
- Created .env.example entries for both mobile and server so setup remains repeatable without committing real keys.

Work Completed - Stripe Card-Only Cleanup

- Constrained the server-created PaymentIntent to payment_method_types: ["card"], preventing Stripe from offering unrelated methods such as bank, Affirm, Cash App Pay, or other dashboard-enabled options inside this early app flow.
- Added PaymentSheet options that hide Link and order card first while the PaymentSheet version was still active.
- Adjusted the Stripe return URL handling for Expo Go versus standalone builds using expo-linking, avoiding hardcoded allavostra:// redirects where Expo Go required a different scheme.
- Disabled Google Pay and Apple Pay inside Expo Go while preserving configuration for future native app-store builds.
- Walked through Stripe dashboard payment-method settings to disable Link in the active/test payment configuration, cleaning up the Stripe test checkout surface.
- Verified that the Stripe sandbox flow reached a completed PaymentIntent and produced the green in-app order confirmation state.

Work Completed - Secure In-App Card Confirmation

- Removed the unwanted second gray Stripe card-entry sheet by replacing `initPaymentSheet/presentPaymentSheet` with `confirmPayment` against the server-created `PaymentIntent` client secret.
- Introduced Stripe-native secure card entry in the Payment overlay so the card information entered in the app is the card information used for the official payment confirmation.
- Kept raw card number, expiration, and CVV out of ordinary app state; the app stores Stripe card-completion and brand information, not the sensitive card data itself.
- Added validation before payment: Debit/Credit Card must be selected, Stripe card details must be complete, billing/name fields must be complete, cart items must exist, and Stripe configuration must be present.
- Built billing details from cardholder/contact/delivery form data and allowed the billing ZIP to be skipped when Billing address matches delivery address is checked.
- Added issuer readout based on Stripe card brand detection, mapping Visa, MasterCard, and American Express into the overlay language.
- After testing `CardField`, switched to `Stripe CardForm` so the payment overlay again resembles the earlier custom design with card number, expiration, and CVV as individual visible rows while retaining Stripe-native secure handling.

Work Completed - Postmark Confirmation Email Backend

- Added `server/api/stripe-webhook.js` as a Vercel route for Stripe webhooks, restricted to POST and configured with `bodyParser` disabled so Stripe signatures can be verified against the raw body.
- Verified incoming Stripe webhook events with `STRIPE_WEBHOOK_SECRET` before doing any email work, returning explicit errors for missing Stripe configuration, missing webhook configuration, missing signature, or invalid signatures.
- Handled `payment_intent.succeeded` by retrieving the `PaymentIntent` from Stripe, checking metadata for `confirmation_email_sent`, and skipping duplicate sends when the confirmation has already been recorded.
- Added `server/lib/postmark.js` to send Postmark email through the official API endpoint using `POSTMARK_SERVER_TOKEN`, `POSTMARK_FROM_EMAIL`, `POSTMARK_REPLY_TO_EMAIL`, and `POSTMARK_MESSAGE_STREAM`.
- Rendered both HTML and plain-text email bodies with customer greeting, order line items, subtotal, delivery, tax, total paid, delivery address, `PaymentIntent` ID, and a short follow-up note.
- Sanitized email, text, line-item, address, and HTML content before sending so data copied from Stripe metadata is trimmed and escaped before it enters email output.
- Updated the `PaymentIntent` metadata after a successful Postmark send with `confirmation_email_sent`, `confirmation_email_message_id`, and `confirmation_email_sent_at` for basic idempotency and traceability.
- Updated server README and `.env.example` to document webhook setup, required event `payment_intent.succeeded`, and Postmark configuration.

Work Completed - Manual Vercel, Stripe, and Postmark Setup

- Walked through importing the GitHub repository into Vercel, selecting the Alla Vostra API project, setting environment variables, deploying the server, and redeploying after new variables were added.
- Confirmed that Vercel hosted the backend at the `alla-vostra-api` project and that the webhook route responded with expected method/signature errors once environment variables

were active.

- Walked through Stripe sandbox setup slowly: API key placement, payment-method configuration, test-mode awareness, completed PaymentIntent inspection, and webhook destination creation.
- Created an active Stripe webhook destination named charming-celebration pointing to /api/stripe-webhook and listening to payment_intent.succeeded.
- Walked through Postmark setup slowly: server/stream navigation, sender confirmation, from/reply-to email settings, message stream choice, server token handling, and Vercel environment-variable entry.
- Verified a Stripe webhook delivery with HTTP 200 and response body {"received": true}.
- Verified Postmark Activity showing processed and delivered records for Your Alla Vostra order confirmation, tagged order-confirmation.
- Verified the confirmation email arrived in Gmail, proving the Stripe-to-Vercel-to-Postmark path worked end to end in the sandbox setup.

Validation and Checks Performed

- Freshly ran node --check mob/app/shop.js before generating this artifact.
- Freshly ran node --check mob/app/_layout.js before generating this artifact.
- Freshly ran node --check mob/utils/stripePayments.js before generating this artifact.
- Freshly ran node --check mob/styles/shopStyles.js before generating this artifact.
- Freshly ran node --check server/api/payment-sheet.js, server/api/stripe-webhook.js, server/api/health.js, and server/lib/postmark.js before generating this artifact.
- Freshly ran npm test --prefix server, which checks the server routes and Postmark helper with node --check.
- Freshly ran git diff --check before generating this artifact; no whitespace or patch-format issues were reported.
- Confirmed the working tree was clean before creating this July 1 combined PDF artifact.
- During the implementation cycle, the mobile payment/CardForm work was checked with git diff --check and an Expo Android export to /tmp/alla-vostra-export-test.
- During the implementation cycle, the deployed Vercel webhook was tested with missing-signature behavior, then with a real Stripe test event delivery returning 200 OK.
- During the implementation cycle, the Postmark path was verified in Postmark Activity and by an actual received Gmail confirmation message.

Files Touched / Added During June 30 and July 1

- Primary Shop implementation, filled-cart grid math, payment overlay state, card form rendering, Stripe confirmPayment flow, billing details, order payload creation, confirmation prompts, and progress-slider behavior: mob/app/shop.js.
- Primary Shop styling, wallet buttons, payment form rows, Stripe card frame/card form sizing, hairlines, checkbox wrapping, issuer readout, and confirmed green center/side-button states: mob/styles/shopStyles.js.
- Stripe provider and app-level Stripe wrapping: mob/app/_layout.js.
- Stripe mobile configuration, return URL, Expo Go handling, backend route URL, and configuration validation: mob/utils/stripePayments.js.
- Expo/Stripe native plugin configuration: mob/app.json.
- Mobile package dependency updates for @stripe/stripe-react-native: mob/package.json and mob/package-lock.json.
- Mobile environment example for public Stripe and backend route values: mob/.env.example.
- Payment method image assets: mob/assets/payments/google-pay-mark.png, mob/assets/payments/apple-pay-mark.png, and mob/assets/payments/paypal-monogram.png.

- Vercel/Stripe server package setup and lockfile: server/package.json and server/package-lock.json.
- Server health and payment routes: server/api/health.js and server/api/payment-sheet.js.
- Stripe webhook and Postmark email implementation: server/api/stripe-webhook.js and server/lib/postmark.js.
- Server setup documentation and environment examples: server/README.md and server/.env.example.
- Generated project state records refreshed during the June 30 work chain: alla_vostra_APP_PROJECT_SNAPSHOT.txt and alla_vostra_WEB_PROJECT_SNAPSHOT.txt.
- This final combined worklog generation creates alla_vostra_worklog_july1.pdf at the repository root.

Important Follow-Up Context

- The Stripe path is now real payment infrastructure, not just front-end UI. Production readiness still requires switching to live Stripe keys, confirming Stripe account/business/tax setup, reviewing live webhook configuration, and keeping test keys out of production builds.
- Card payments are secure because card details are handled by Stripe-native CardForm/confirmPayment, but the app is not yet a full ordering system with database persistence, admin order management, refunds UI, delivery dispatch, or customer support tooling.
- Google Pay, Apple Pay, and PayPal are still not equivalent to the card path. They should not be represented as fully active production payment methods until each is implemented and tested through the proper provider flow.
- Postmark currently works in the configured test/approval state. Before sending confirmation emails to arbitrary real customers, Postmark approval/domain authentication and sender reputation details should be completed.
- The Stripe webhook is intentionally idempotent at the PaymentIntent metadata level, but a more mature order system should eventually store order records and email status in a database rather than relying only on PaymentIntent metadata.
- The payment overlay remains visually sensitive. The CardForm gives Stripe security while approximating the old split-field design, but exact native rendering can vary on Android, iOS, Expo Go, and production builds.
- The filled-cart 2x2 grid remains layout-sensitive. Any future asset, scrollbar, divider, or margin changes should be tested with one-entry and multi-entry carts.
- Vercel environment variables now matter for live behavior. After adding or changing them, a redeploy is required before the deployed functions see the new values.

Output Artifact

- Created this combined worklog PDF: alla_vostra_worklog_july1.pdf.
- The file is named after July 1 because it combines June 30 and July 1 work and follows the existing root-level worklog naming convention.
- No external PDF package was installed; this artifact was generated with a small standard-library PDF writer in the same plain-text-readable spirit as the prior daily worklogs.